

**Department of Electrical Engineering  
The Superior University, Lahore**

**Programming Fundamentals**

**Experiment No.2**

**Introduction of Compilers, Program Structure, Input-Output, Data Types, Comments.**

**Prepared for:**

**Eisha Nawaz**

**By:**

**Name:** \_\_\_\_\_

**ID:** \_\_\_\_\_

**Section:** \_\_\_\_\_

**Semester:** \_\_\_\_\_

**Total Marks:** \_\_\_\_\_ 10

**Obtained Marks:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

**Date:** \_\_\_\_\_

## Experiment No.2

**Introduction of Compilers, Program Structure, Input-Output, Data Types, Comments.**

**(Rubrics)**

**Name:** \_\_\_\_\_

**ID:** \_\_\_\_\_

### A. PSYCHOMOTOR

| Serial No.       | Criteria                   | Allocated Marks | Unacceptable 0%                                    | Poor 25% | Fair 50% | Good 75% | Excellent 100% | Total Obtained |
|------------------|----------------------------|-----------------|----------------------------------------------------|----------|----------|----------|----------------|----------------|
| 1                | Software learning          | 2               | 0                                                  | 0.5      | 1        | 1.5      | 2              |                |
| 2                | Follow Procedures          | 2               | 0                                                  | 0.5      | 1        | 1.5      | 2              |                |
| 3                | Accuracy in Output Results | 2               | 0                                                  | 0.5      | 1        | 1.5      | 2              |                |
| <b>Sub-Total</b> |                            | <b>6</b>        | <b>Sub Total Marks Obtained in Psychomotor (P)</b> |          |          |          |                |                |

### B. AFFECTIVE

| Sr. No.          | Criteria      | Allocated Marks | Unacceptable 0%                                  | Poor 25% | Fair 50% | Good 75% | Excellent 100% | Total Obtained |
|------------------|---------------|-----------------|--------------------------------------------------|----------|----------|----------|----------------|----------------|
| 1                | Lab Report    | 2               | 0                                                | 0.5      | 1        | 1.5      | 2              |                |
| 2                | Assigned Task | 2               | 0                                                | 0.5      | 1        | 1.5      | 2              |                |
| <b>Sub Total</b> |               | <b>4</b>        | <b>Sub Total Marks Obtained in Affective (A)</b> |          |          |          |                |                |

**Instructor Name:** Miss Eisha Nawaz

**Total Marks (P+A):** 10

**Instructor Signature:** \_\_\_\_\_

**Obtained Marks (P+A):** \_\_\_\_\_

### CONTENTS OF LAB 2

---

|       |                                   |    |
|-------|-----------------------------------|----|
| 1     | Objectives:                       | 5  |
| 2     | Introduction of Compiler:         | 5  |
| 2.1   | Using Dev-C++                     | 5  |
| 2.1.1 | Create a Source File              | 6  |
| 2.1.2 | Save the Program                  | 6  |
| 2.1.3 | Files Created                     | 7  |
| 2.1.4 | Writing Source Code               | 8  |
| 2.1.5 | Compilation and Link              | 8  |
| 2.1.6 | Compilation and Execution         | 8  |
| 3     | Structure of a Program            | 9  |
| 3.1   | Example 1                         | 9  |
| 4     | Basic Data Types                  | 10 |
| 5     | Comments                          | 11 |
| 5.1   | Types of Comments                 | 11 |
| 5.1.1 | Example 1                         | 11 |
| 5.1.2 | Example 2                         | 11 |
| 6     | Basic Input and Output Operations | 12 |
| 6.1   | Output                            | 12 |
| 6.1.1 | Escape Sequences                  | 12 |
| 6.2   | Input                             | 12 |
| 6.2.1 | Example 1                         | 12 |
| 7     | Utilization of Variables          | 13 |
| 7.1   | Example 1                         | 13 |
| 7.2   | Example 2                         | 14 |
| 8     | Lab Tasks                         | 14 |
| 8.1   | Task 1                            | 15 |
| 8.2   | Task 2                            | 15 |
| 8.3   | Task 3                            | 15 |
| 9     | Home Assignment                   | 15 |
| 9.1   | Task 1                            | 15 |
| 9.2   | Task 2                            | 16 |
| 9.3   | Task 3                            | 16 |

## Introduction of Compilers, Program Structure, Input Output, Data Types, Comments, and Arithmetic Operators

### 1 OBJECTIVES:

---

- To briefly introduce Dev-C++ IDE (integrated development environment)
- Write and compile a basic C++ program.
- Learn about structure of a C++ program.
- Use input/output statements.
- Work with basic data types and comments.
- To use mathematical operators

### 2 COMPILED LANGUAGES AND C++:

---

#### 2.1 WHY USE A LANGUAGE LIKE C++?

At its core, a computer is just a processor with some memory, capable of running tiny instructions like “store 5 in memory location 23459.” Why would we express a program as a text file in a programming language, instead of writing processor instructions?

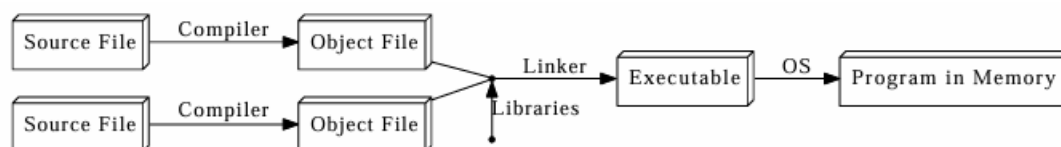
The advantages:

1. **Conciseness:** programming languages allow us to express common sequences of commands more concisely. C++ provides some especially powerful shorthands.
2. **Maintainability:** modifying code is easier when it entails just a few text edits, instead of rearranging hundreds of processor instructions. C++ is object oriented, which further improves maintainability.
3. **Portability:** different processors make different instructions available. Programs written as text can be translated into instructions for many different processors; one of C++’s strengths is that it can be used to write programs for nearly any processor.

C++ is a high-level language: when you write a program in it, the shorthands are sufficiently expressive that you don’t need to worry about the details of processor instructions. C++ does give access to some lower-level functionality than other languages (e.g. memory addresses).

#### 2.2 COMPILATION PROCESS

A program goes from text files (or source files) to processor instructions as follows:



Object files are intermediate files that represent an incomplete copy of the program: each

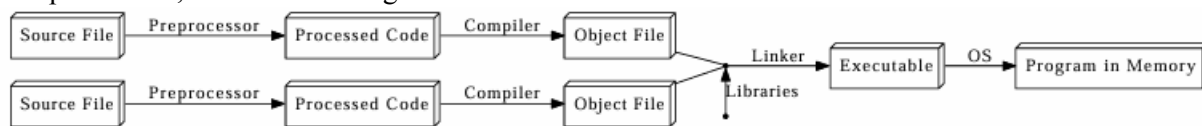
source file only expresses a piece of the program, so when it is compiled into an object file, the object file has some markers indicating which missing pieces it depends on. The linker takes those object files and the compiled libraries of predefined code that they rely on, fills in all the gaps, and spits out the final program, which can then be run by the operating system(OS).

The compiler and linker are just regular programs. The step in the compilation process in which the compiler reads the file is called **parsing**.

In C++, all these steps are performed ahead of time, before you start running a program.

In some languages, they are done during the execution process, which takes time. This is one of the reasons C++ code runs far faster than code in many more recent languages.

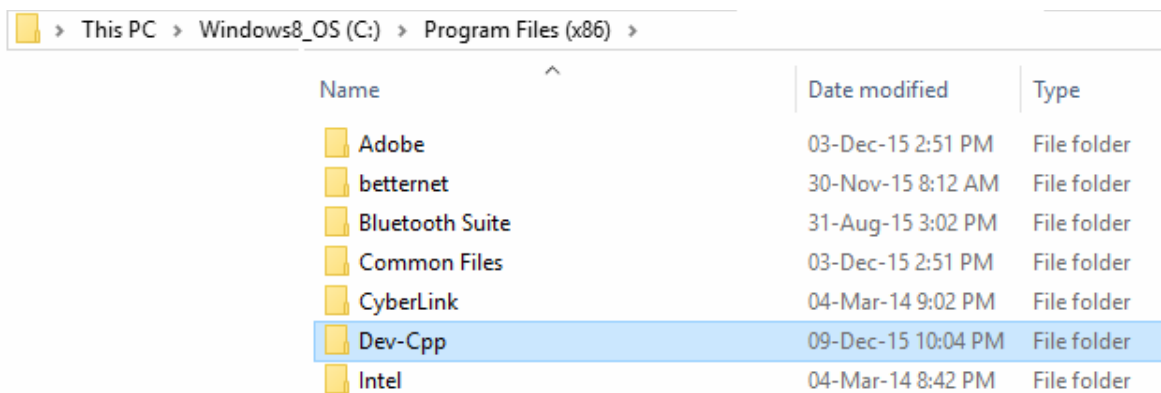
C++ actually adds an extra step to the compilation process: the code is run through a **preprocessor**, which applies some modifications to the source code, before being fed to the compiler. Thus, the modified diagram is:



## Note:

Everything in C++ is case sensitive: someName is not the same as SomeName.

1. To start Dev-C++, go to My Computer, Program Files, Dev-Cpp folder as shown in *Figure 1*.



*Figure 1 Dev-C++ Opening*

2. Click on devcpp.exe file

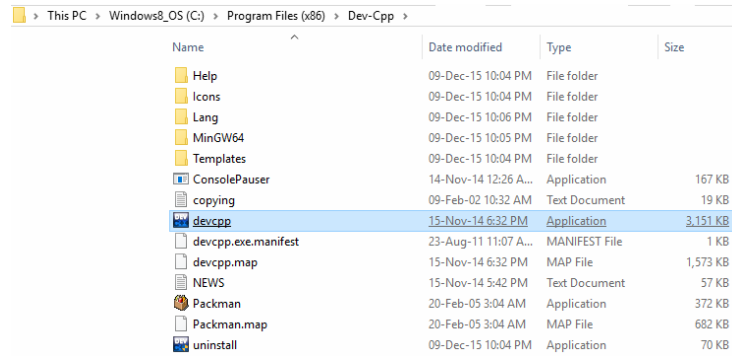


Figure 2 devcpp.exe

## 2.2.1 Create a Source File

Before creating the source code file, it is necessary to create a Source File (File > New > Source File).

## 2.2.2 Save the Program

Figure 3 show the process of creating a file.

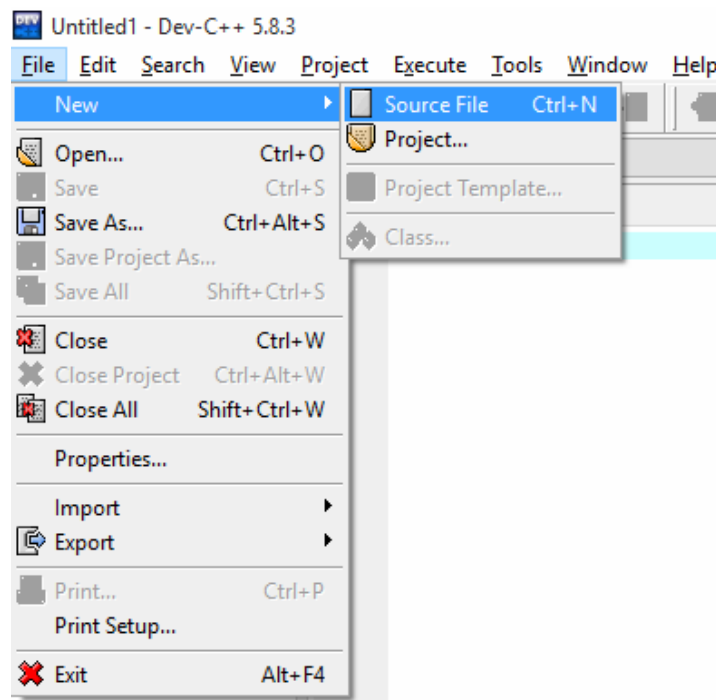


Figure 3 Creating a Source File

Figure 4 shows the steps of saving a file.

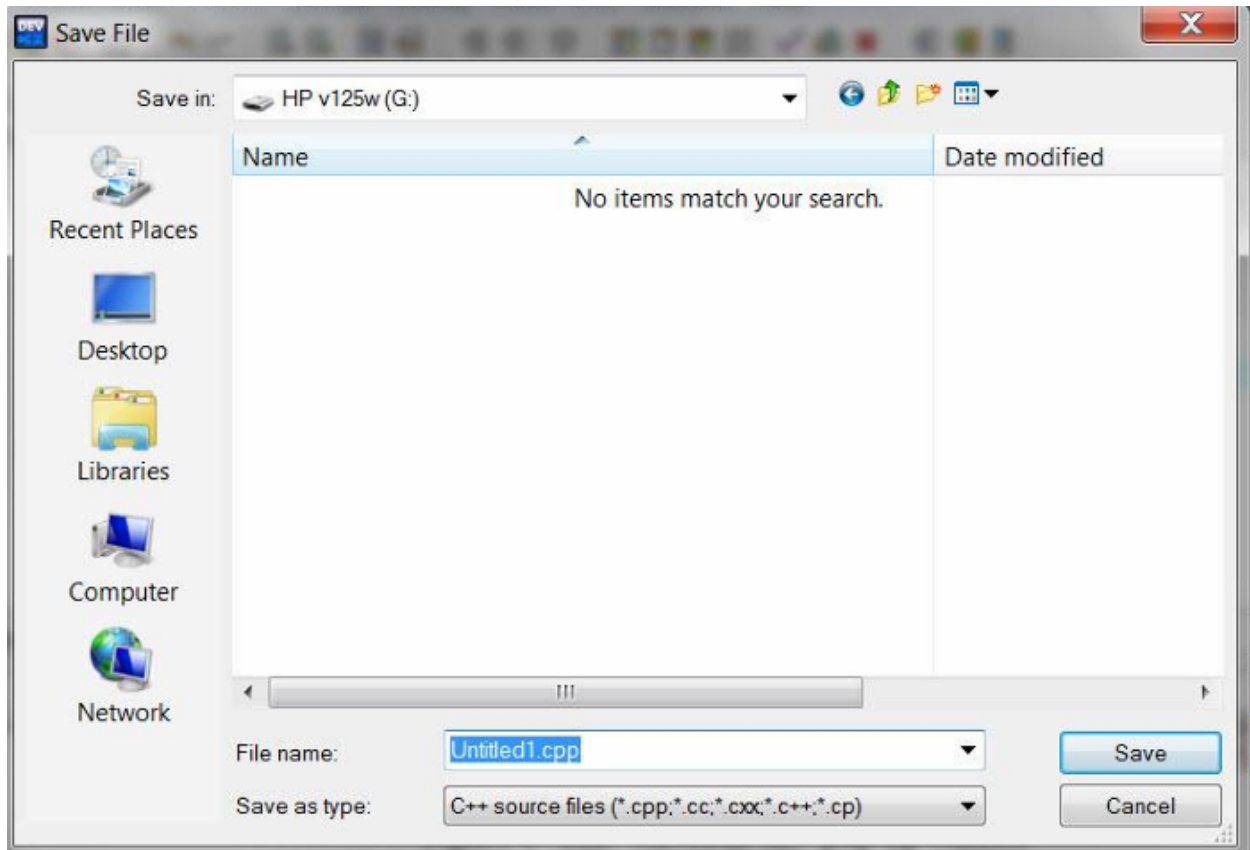


Figure 4 Saving File

When you name your program, you must specify three things:

- (a) the drive name (nothing, if saving on the hard drive; d: or e: or something else if you have a flash drive)
- (b) the name of the program itself, which should be something like prog1 or first or assn1
- (c) the extension, which must be .cpp for a C++ program.

After indicating the folder where the Source file (.cpp) will be saved, the IDE generates a basic source code file (by default, main.c). These files are not saved in the project folder until the programmer saves or compiles the program.

Figure 5 highlights the on-screen area of File Explorer, Source Code Editor, and Results of the Compiler as portrayed in DevC++.

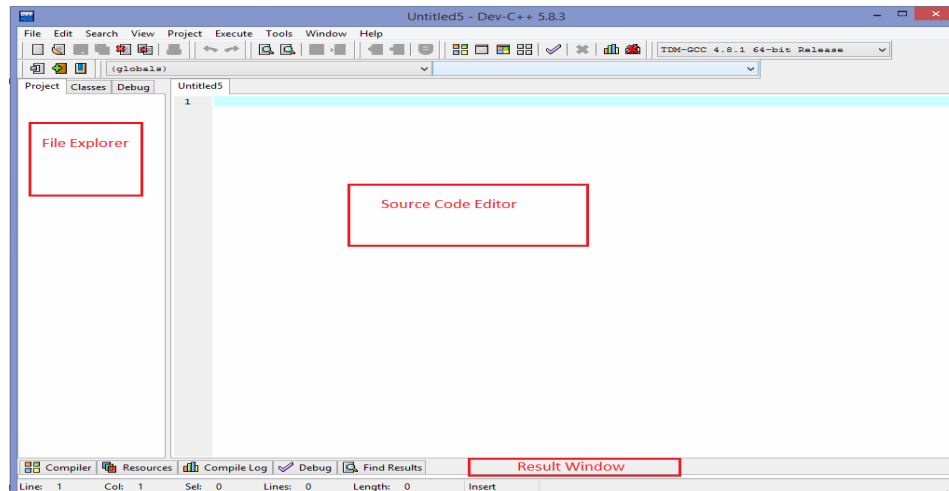


Figure 5 File Explorer, Source Code Editor & Result of Compiler

Compile your program by selecting Execute | Compile or by clicking on this icon  above the edit window. If your compilation is successful, a Done message will appear; click OK to return to the editor. If your compilation is unsuccessful, an error message will appear; click OK to view a list of errors that you must correct. Use the editor to correct the errors, and repeat step 4 until you are successful.

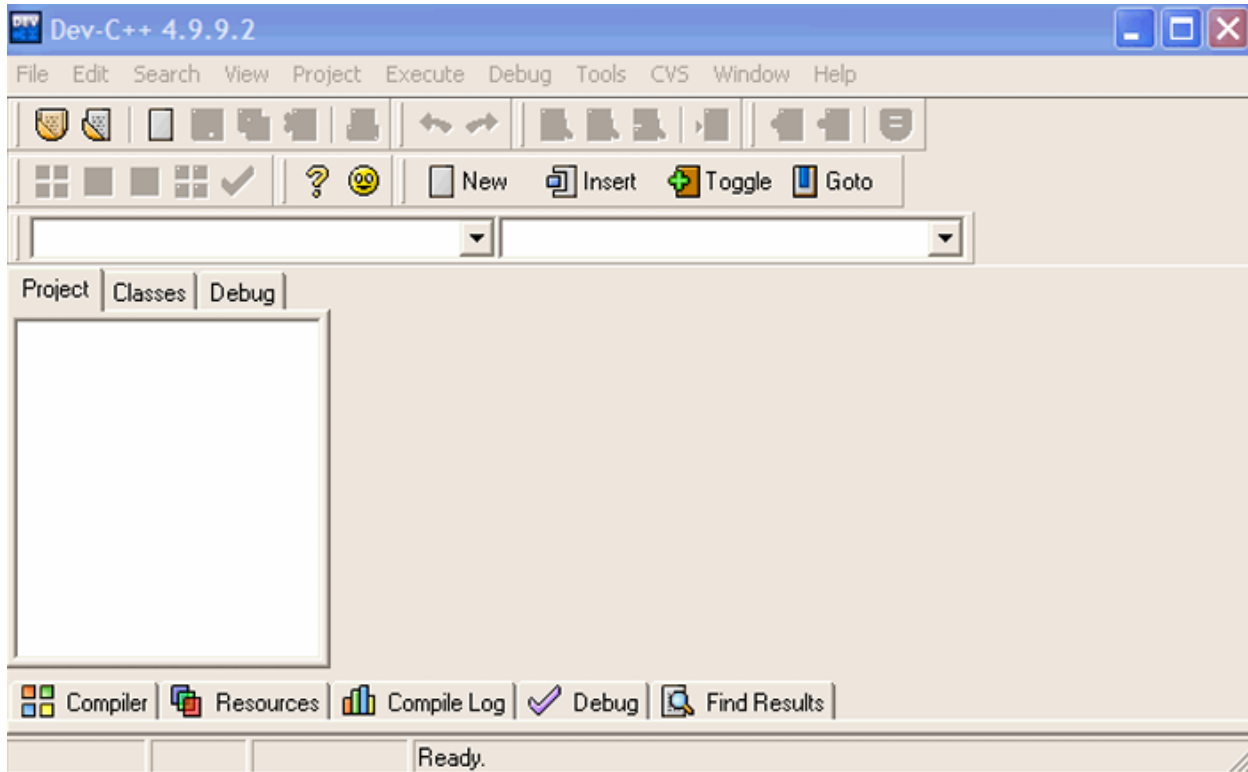
Execute your program by selecting Execute | Run or by clicking on this icon: 

Print your program by selecting File | Print or by clicking this icon.  In the window that pops up, remove any check mark in the box that says “Line numbers.” (With line numbers, your printed lines may run off the page.) Click OK.

## 2.2.3 Entering Dev-C++

A shortcut to Dev-C++ has been placed on the desktop. It may be in a folder labeled Programming Languages. Double-click on the folder to open it, find the icon shown above, and double-click on it. In other locations, perhaps at home, to start Dev-C++, you must find it in the Start menu. A typical way to find it is to select Start | All Programs | DevC++. You can also search for the program using the Search box, typing in Dev-C++.

After you start the program, the Dev-C++ environment, shown in Figure, will appear on your screen. (These screens are from an earlier version of Dev-C++ but are identical except for the version number, which is now 5.3.0.3.) The Dev-C++ main menu selections appear across the top of the screen. Placing the cursor over an icon will cause its use to be displayed at the bottom left of the Dev-C++ window.



For example, placing the cursor on top of the second icon from the left causes the words

"Open Project or File" to appear below the icon.

After creating a New file as you type in your program, you will notice that Dev-C++ uses different colors for different elements of your program; this is called syntax highlighting. The colors can be changed, or you can choose to have everything displayed in one color. Syntax highlighting can help you in debugging your program; for example, if you have typed in a keyword and it doesn't appear in boldface, the word is probably misspelled. If you have forgotten to terminate a comment, all the text following it will be in the same color as the comment.

The IDE window includes three sub-windows: **The Project Files Explorer**, the **Result Window**, and the **Source Code Editor**. These windows can be resized and minimized. The Files Explorer window shows the name of the project and the included files. The Project tab usually contains a single file with the source code of the program. In this pane, we can find two additional tabs: Classes and Debug. Classes tab shows the functions of the program. Debug tab shows watched variables in the debugging process. The Results window is used to present the results of the actions of the IDE: compilation errors, compiling directives, debugging commands, etc.

## 2.2.4 WE RECOMMEND THAT YOU SAVE YOUR FILE EVERY FIVE OR TEN MINUTES.

If you do not save your file, and the computer crashes or someone kicks out the plug, you will lose all your unsaved work. It is better to lose less work by saving every few minutes.

Dev-C++ automatically saves your file before compiling it. This can cause two surprises:

1) the Save File window may appear asking you to name the file, and 2) if you are making some "what if I do this" kinds of changes, you may lose the original version of your program. If you want to keep more than one version of a program, use the File | Save as feature to save the file with a different name.

## 2.2.5 Files Created

The files of the project are saved when the source code file is saved. The remaining files of the project are saved when the application is compiled and stored in the project folder.

Table 1 Description of Files and Their Extensions

| Files            | Extension | Description                                                                                                         |
|------------------|-----------|---------------------------------------------------------------------------------------------------------------------|
| Project file     | .dev      | Project configuration data                                                                                          |
| Make file        | .win      | Required for the compilation process. Manages program dependencies and includes instructions for the linker         |
| Source code file | .cpp      | Source code                                                                                                         |
| Object file      | .o        | Object code resulting from the compilation of the source code. Each .c has a corresponding .o after the compilation |
| Executable file  | .exe      | Executable application                                                                                              |

## 2.2.6 Writing Source Code

Once the project has been created, we can start writing our C++ program. It is recommended to use the classic color configuration of the editor (**Tools > Editor options > Syntax > Color Speed Settings > Classic**) and to activate the support for opening and closing brackets (**Tools > Editor options > General > Highlight matching braces / parenthesis**). The editor highlights with different colors keywords and other elements of the C++ language. The classic scheme uses:

- Light blue for comments
- Green for included libraries
- Red for text strings
- Bold black for C keyword

## 2.2.7 Compilation and Link

To run a program, the source code must be compiled and linked. Dev-C++ performs the complete process by clicking the Compile button (or Ctrl + F9). While the compilation and link process is being performed, the IDE shows a dialog with related information. If the process is successful, the window shows the message Done.

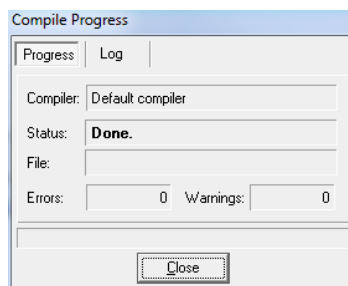


Figure 6 Compiler Progress

## 2.2.8 Compilation and Execution

To run a program, the source code must be compiled and linked. Codeblocks performs the complete process by clicking the Compile button (or Ctrl + F9). While the compilation and link process is being performed, the IDE shows a dialog with related information. If the process is successful, a console window is prompted and result is displayed.

If your program is correct---and if you remembered to include the line `system("pause");` right before `return 0;` --a window will appear in which you can review the results, or output, from running your program. Look at the output from running your program, to see that the program did what you expected it to do. If it did not, correct any mistakes that you can identify, then recompile and rerun the program. (You must close the output window before you can run your program again.)

Errors may be discovered as your program is running; these are called **execution errors**. Some execution errors may be identified by error messages in the Message window. Other errors may show up by an absence of output, or by output which is not correct. Some errors show up because the computer never stops executing the program (this is called an infinite loop). Once again, if you cannot identify the errors, print the program and the output and seek help.

If the program ran correctly and gave the correct output, make a copy of the program and the output to turn in to your instructor for grading. **Do not erase your file from your disk.** If your instructor finds any errors, you may have to modify and rerun the program. In addition, you should save all graded programs until the end of the semester in case your grade is lost.

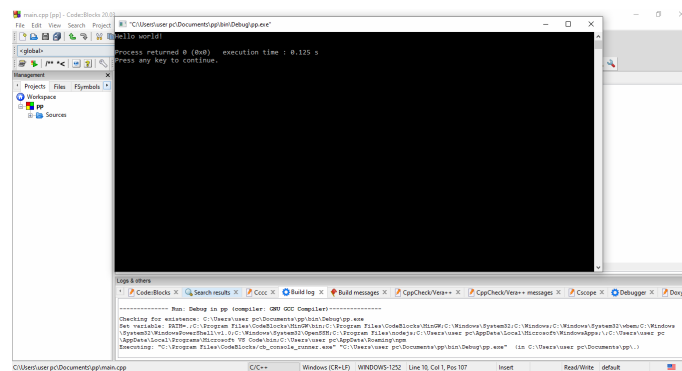


Figure 7 Compiler Progress

## 2.2.9 What to Do About an Infinite Loop, or Worse

Your program might have an infinite loop or, worse, might cause the machine to freeze or hang, which means it will stop responding to your commands. Here are some suggestions of what to do if something like this happens.

- (1) Breaking out of an infinite loop which prints is easy: simply close the output window, and the program will stop running.
- (2) If that doesn't work, try to break out of the program by pressing **<Ctrl>-<Break/Pause>**. If that doesn't work, try **<Ctrl>-C**. You may have to do this several times.
- (3) If that doesn't work, try to end the task by pressing the **<Ctrl>**, **<Alt>** and **<Delete>** keys

together. The Task Manager will open, and you can use the mouse to highlight the name of the offending program, then select "End task" to terminate it.

### 3 STRUCTURE OF A PROGRAM

---

Probably the best way to start learning a programming language is by writing a program. Here, it should be kept in mind that “dev C++” compiler is used for compilation and execution of codes.

Let us begin with our first program:

#### 3.1 EXAMPLE 1

Table 2 portrays an exemplary program. The left column of the table shows the lines of codes of the program while right column shows the output of the program.

*Table 2 Example with namespace*

|   | Lines of Code                                    | Result                                  |
|---|--------------------------------------------------|-----------------------------------------|
| 1 | //Lab_1_Example_1.cpp                            | <b>Welcome to Superior University!!</b> |
| 2 | #include<iostream>                               |                                         |
| 3 | using namespace std;                             |                                         |
| 4 | int main ()                                      |                                         |
| 5 | {                                                |                                         |
| 6 | Cout<<" <b>Welcome to Superior University!</b> " |                                         |
| 7 | return 0;                                        |                                         |
| 8 | }                                                |                                         |

*Table 2 Example without namespace*

|   | Lines of Code                                         | Result                                 |
|---|-------------------------------------------------------|----------------------------------------|
| 1 | //Lab_1_Example_1.cpp                                 | <b>Welcome to Superior University!</b> |
| 2 | #include<iostream>                                    |                                        |
| 4 | int main ()                                           |                                        |
| 5 | {                                                     |                                        |
| 6 | Std::Cout<<" <b>Welcome to Superior University!</b> " |                                        |
| 7 | return 0;                                             |                                        |
| 8 | }                                                     |                                        |

The double colon :: is the scope resolution operator, and indicates that you want to access the cout object declared in the std namespace.

The first panel (white) shows the source code for our program. The second panel (gray) shows the result of the program once compiled and executed. To the left, the grey numbers represent the line numbers - these are not part of the program, and are shown here only for informational purposes.

#### Line 1: //Lab\_1\_Example\_1.cpp

This is a comment line.

**Line 2: #include <iostream>**

Lines beginning with a hash sign (#) are **directives** for the **preprocessor**. They are not regular code lines with expressions but indications for the compiler's preprocessor. In this case the directive #include <iostream> tells the preprocessor to include the iostream header file. This specific file (iostream) includes the declarations of the basic standard input-output library in C++, and it is included because its functionality is going to be used later in the program.

**Line 3: using namespace std;**

All the elements of the standard C++ library are declared within what is called a **namespace**, the namespace with the name std. So in order to access its functionality we declare with this expression that we will be using these entities. This line is very frequent in C++ programs that use the standard library, and in fact it will be included in most of the source codes included in this lab manual.

**Line 4: int main ()**

This line corresponds to the beginning of the definition of the main function. The main function is where all C++ programs start their execution, independently of its location within the source code. It does not matter whether there are other functions with other names defined before or after it - the instructions contained within this function's definition will always be the first ones to be executed in any C++ program. For that same reason, it is essential that all C++ programs have a main function. The word main is followed in the code by a pair of parentheses (). These parentheses may enclose a list of parameters within them. We will discuss parameters soon with functions.

**Line 5: braces {}**

The braces, {}, indicates a code block; in this case, the C++ code is for the function main. We know that this is a function because of the basic format: first, there is the type of the return value, then the name of the function with a pair of parentheses, which is used to declare the parameters passed to the function (and their types). In this example, the function is called main and the parentheses are empty, indicating that the function has no parameters. The identifier before the function name (int) says that the function will return an integer.

The convention with C++ is that a function called main is the **entry point** of the executable, that is, when you call the executable from the command line, this will be the first function in your code that will be called.



This simple example function immediately immerses you into an aspect of C++ that irritates programmers of other languages: the language may have rules, but the rules don't always appear to be followed. In this case, the main function is declared to return an integer, but the code returns no value. The rule in C++ is that, if the function declares that it returns a value, then it must return a value. However, there is a single exception to this rule: if the main function does not return a value, then a value of 0 will be assumed. C++ contains many quirks such as this, but you will soon learn what they are and get used to them

**Line 6: cout<<"Welcome to Superior University!";**

This line is a C++ instruction. An instruction is a simple or compound expression that can actually produce some effect. This statement will print Welcome to Superior University! on output screen.

**cout** is the name of the standard print instruction in C++, and the meaning of the entire instruction is to print a sequence of characters (in this case **the Welcome to Superior University!**) into the standard output screen.

Notice that the statement ends with a semicolon character (;). In C++ every statement must have a semicolon at the end of line.

**Line 7: return 0;**

The return statement is use to end main function.

## 4 BASIC DATA TYPES

---

Here are some basic data types that we will mostly be using in our Lab sessions.

*Table 3 Data Types*

| Data Types               | Description                                  | Size    |
|--------------------------|----------------------------------------------|---------|
| <b>char</b>              | Character or small integer.                  | 1byte   |
| <b>short int (short)</b> | Short Integer.                               | 2bytes  |
| <b>int</b>               | Integer.                                     | 4bytes  |
| <b>long int (long)</b>   | Long integer.                                | 8bytes  |
| <b>float</b>             | Floating point number.                       | 4bytes  |
| <b>double</b>            | Double precision floating point number.      | 8bytes  |
| <b>long double</b>       | Long double precision floating point number. | 16bytes |

## 5 COMMENTS

---

A comment is a line or paragraph of text in a file such that the line or paragraph is not considered when the compiler (or the parser) is decoding the code of the file.

### 5.1 TYPES OF COMMENTS

The C/C++ language accepts two types of comments:

- To write a comment on one line, type two forward slashes // and type the comment. Anything on the right side of both forward slashes would not be read by the compiler (actually the parser).

#### 5.1.1 Example 1

Single line can be commented by using // as shown below:

```
// My First Program
// .....
```

- To write a comment paragraph, start it with a forward slash / followed by an asterisk \*, write the comment. To end the comment, type an asterisk \* followed by a forward slash /

```
/* This is an example of a forward slash asterisk comment */
```

### 5.1.2 Example 2

Several lines can also be commented as shown below:

```
/*
cout<< "\n\n Hello to our project!";
getch ();
*/
```

## 6 BASIC INPUT AND OUTPUT OPERATIONS

---

### 6.1 OUTPUT

Display devices like LCD and CRT are used to show information to user. Displaying information is nothing to do with processing of data. In Dev C++ we use **cout** to display information on display screen. Its instruction structure is:

Table 7

| Keyword     | Output Stream Operator | String or Variable Name | ; |
|-------------|------------------------|-------------------------|---|
| <b>Cout</b> | <<                     | "Hello World"           | ; |
| <b>Cout</b> | <<                     | endl                    | ; |

#### 6.1.1 Escape Sequences

These are non-printing characters. They are special character set, each with specific meaning. An escape sequence always begins with a back slash and is followed by one or more special characters. These characters are used to alter the behavior of output.

Table 8 Escape Sequences

| Escape Sequence | Meaning         |
|-----------------|-----------------|
| <b>\n</b>       | New Line        |
| <b>\t</b>       | Horizontal Tab  |
| <b>\a</b>       | Alert(Bell)     |
| <b>\\</b>       | Back Slash      |
| <b>\"</b>       | Quotation Mark  |
| <b>\f</b>       | Form feed       |
| <b>\r</b>       | Carriage Return |
| <b>\0</b>       | Null            |

## 6.2 INPUT

Nowadays, there is no program without user input. Especially keyboard is most commonly used as an input device. C++ use “**cin**” to take input from user. It will keep taking input until you hit **Return Key**. But put **Null** when **space bar** is pressed. So using **cin** we can only take input of a numeric value or a single word. Its instruction structure is:

Table 9 Input Stream Operator

| Keyword    | Input Stream Operator | Variable Name | ; |
|------------|-----------------------|---------------|---|
| <b>Cin</b> | >>                    | Input         | ; |
| <b>Cin</b> | >>                    | A             | ; |

### 6.2.1 Example 1

As shown in the given table, two variables are taken as input and both the variables are summed up using the addition operator.

Table 10 Input Example

|    | Lines of Code                         | Result                                                                          |
|----|---------------------------------------|---------------------------------------------------------------------------------|
| 1  | //Lab_1_Example_4.cpp                 | Please provide value of a: 12<br>Please provide value of b: 16<br>Result is: 28 |
| 2  | #include <iostream>                   |                                                                                 |
| 3  | using namespace std;                  |                                                                                 |
| 4  | int main ()                           |                                                                                 |
| 5  | {                                     |                                                                                 |
| 6  | int a, b;                             |                                                                                 |
| 7  | int result;                           |                                                                                 |
| 8  | cout<< “Please provide value of a: ”; |                                                                                 |
| 9  | cin>> a;                              |                                                                                 |
| 10 | cout<< “Please provide value of b: ”; |                                                                                 |
| 11 | cin>> b;                              |                                                                                 |
| 12 | result = a + b;                       |                                                                                 |
| 13 | cout<< “Result is: ”<<result;         |                                                                                 |
| 14 | return 0;                             |                                                                                 |
| 15 | }                                     |                                                                                 |

## 7 UTILIZATION OF VARIABLES

**Declaration:** in variable declaration first we have to provide the **Data Type** and the **Name** of variable.

**Initialization:** in variable initialization we have to provide a starting value, otherwise C++ will auto initialize it with a garbage value.

Syntax for variable declaration and initialization in C++ is shown in the given table:

Table 4 Declaration and Initialization of the Variables

| Data Type  | Space | Variable Name<br>(Identifier) | = | Initial Value | ; |
|------------|-------|-------------------------------|---|---------------|---|
| <b>int</b> |       | A                             | = | 10            | ; |

## 7.1 EXAMPLE 1

The given example shows the use of different data types:

Table 5 Example 1

|    | Lines of Code                       | Result                                              |
|----|-------------------------------------|-----------------------------------------------------|
| 1  | //Lab_1_Example_2.cpp               | Marks are = 56<br>Average is = 40.3<br>Grade is = B |
| 2  | #include <iostream>                 |                                                     |
| 3  | using namespace std;                |                                                     |
| 4  | int main ()                         |                                                     |
| 5  | {                                   |                                                     |
| 6  | int Marks = 56;                     |                                                     |
| 7  | float Avg = 40.3;                   |                                                     |
| 8  | char Grade = 'B';                   |                                                     |
| 9  | cout<< "Marks are = "<<Marks<<endl; |                                                     |
| 10 | cout<< "Average is = " <<Avg<<endl; |                                                     |
| 11 | cout<< "Grade is = " <<Grade<<endl; |                                                     |
| 12 | return 0;                           |                                                     |
| 13 | }                                   |                                                     |

## 7.2 EXAMPLE 2

The given example performs an increment on one integer and then it performs arithmetic operations on both variables:

Table 6 Example 2

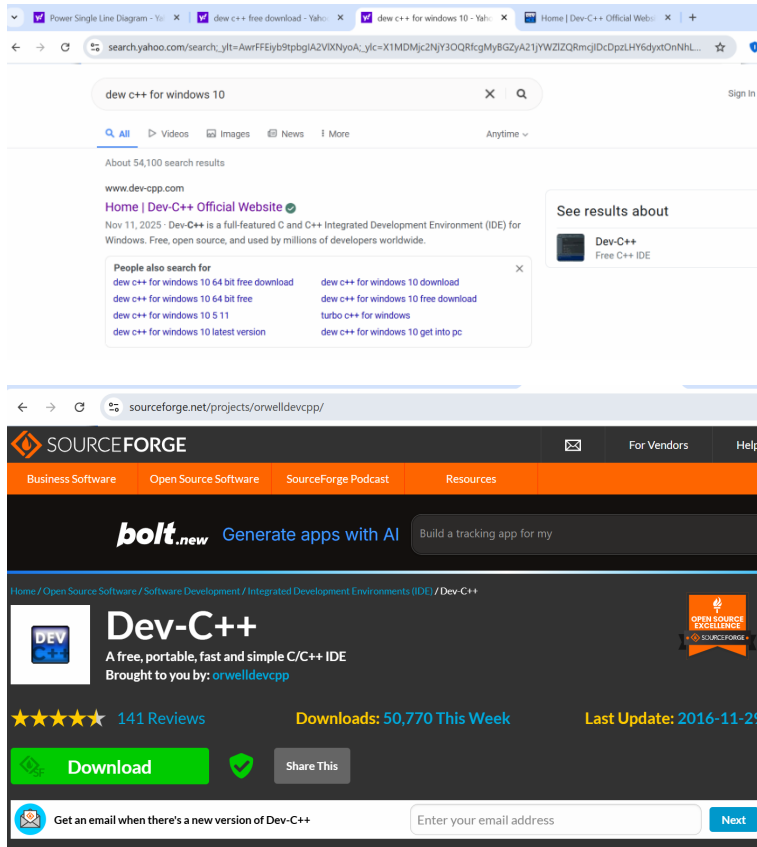
|    | Lines of Code             | Result |
|----|---------------------------|--------|
| 1  | //Lab_1_Example_3.cpp     | 4      |
| 2  | #include <iostream>       |        |
| 3  | using namespace std;      |        |
| 4  | int main ()               |        |
| 5  | {                         |        |
| 6  | // declaring variables:   |        |
| 7  | int a, b;                 |        |
| 8  | int result;               |        |
| 9  | // process:               |        |
| 10 | a = 5;                    |        |
| 11 | b = 2;                    |        |
| 12 | a = a + 1;                |        |
| 13 | result = a - b;           |        |
| 14 | // print out the result:  |        |
| 15 | cout<< result;            |        |
| 16 | // terminate the program: |        |
| 17 | return 0;                 |        |
| 18 | }                         |        |

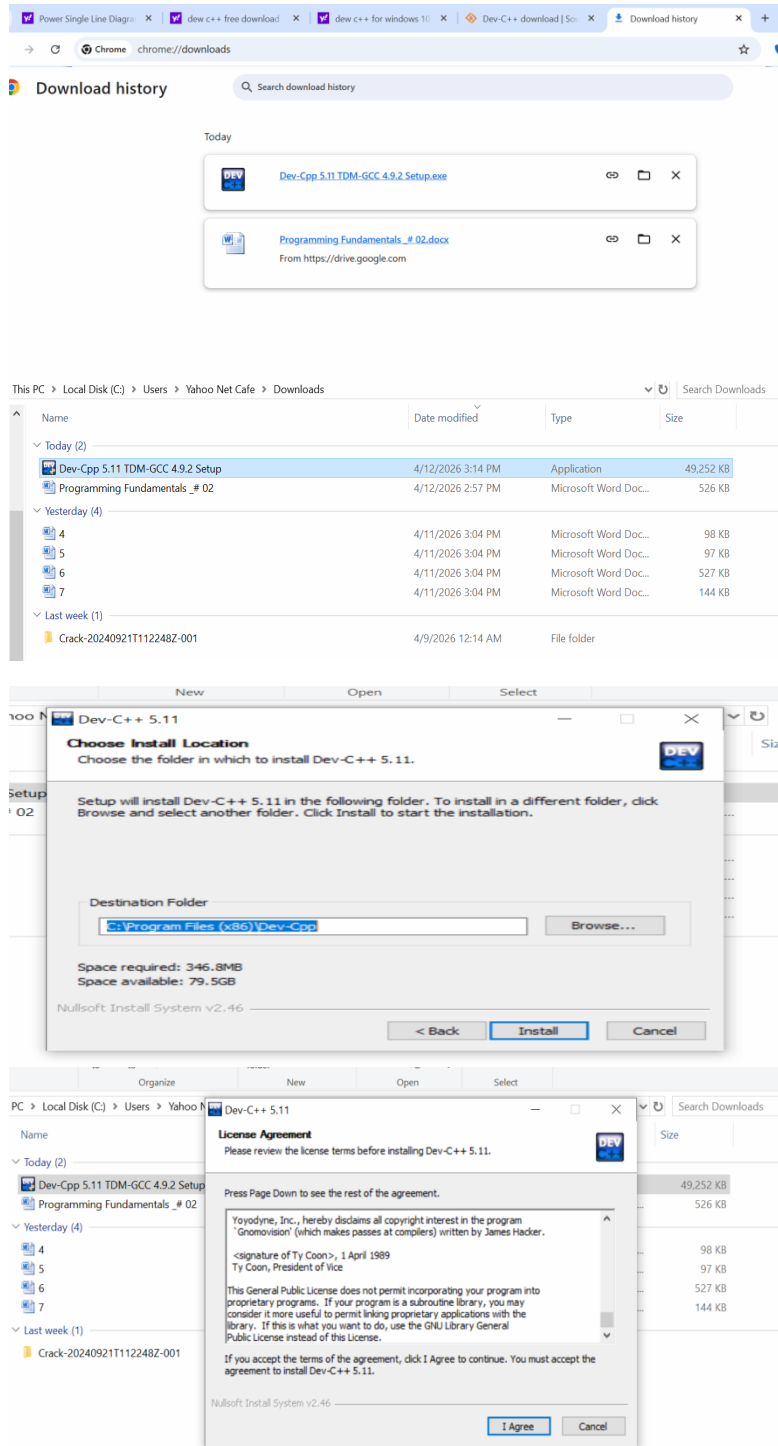
## 8 LAB TASKS

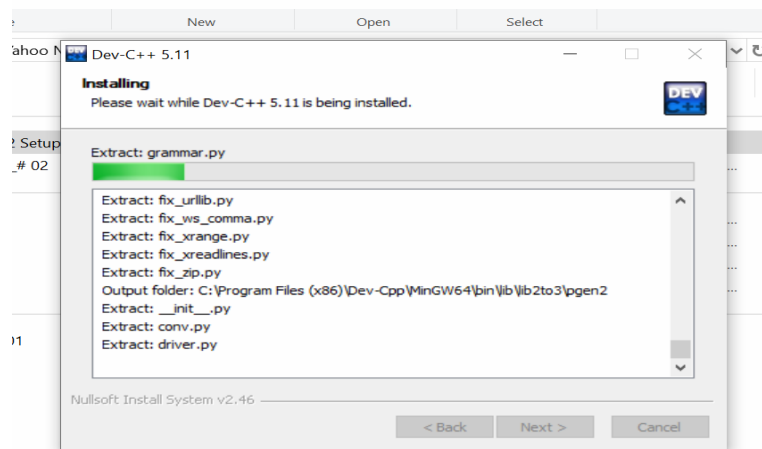
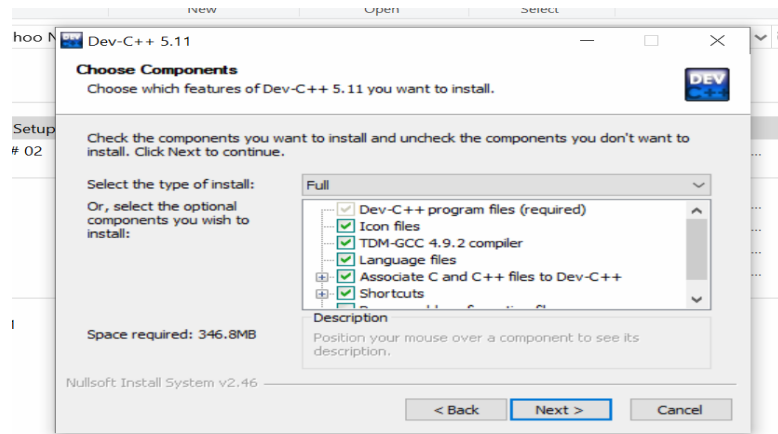
Solve the following tasks.

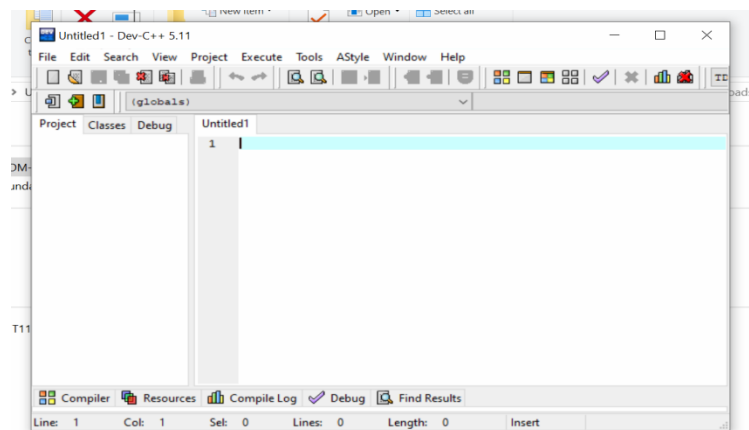
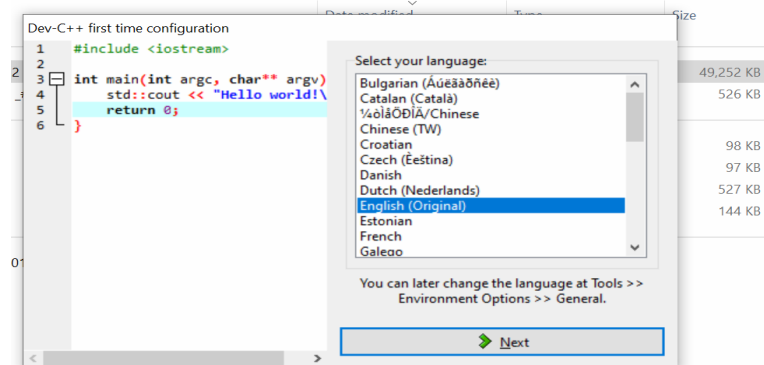
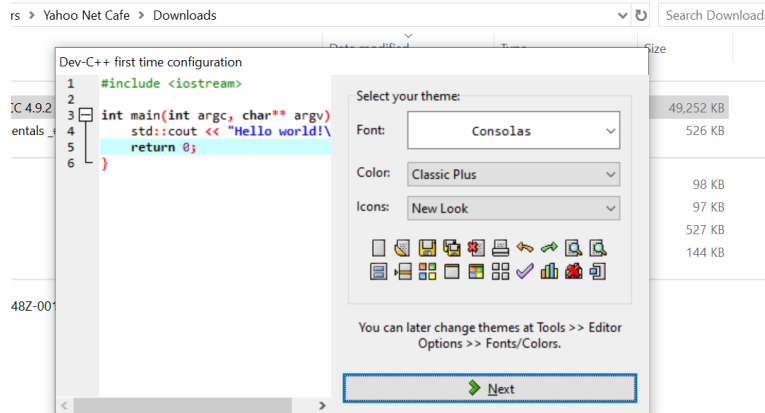
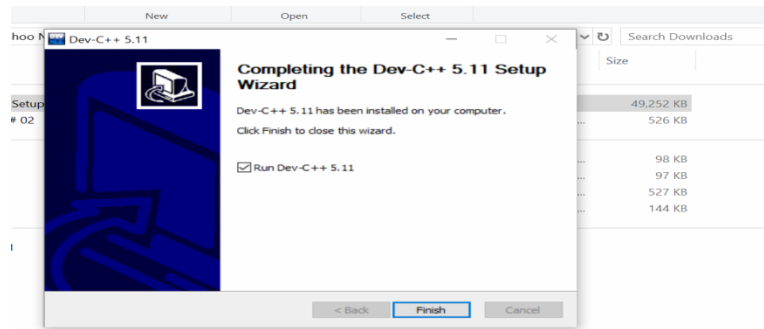
## 8.1 TASK 1

Download and Install the Dev c++ tool and attach each step Screenshot with one line description here.









## 8.2 TASK 2

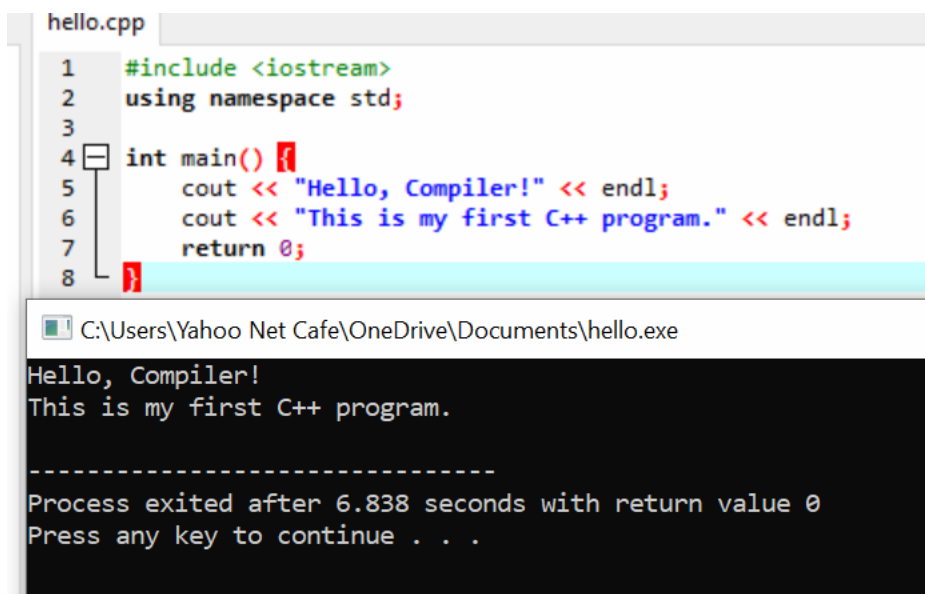
1. Create a new C++ file named hello.cpp.

2. Write a program that prints:

    Hello, Compiler!

    This is my first C++ program.

3. Compile and run your program.



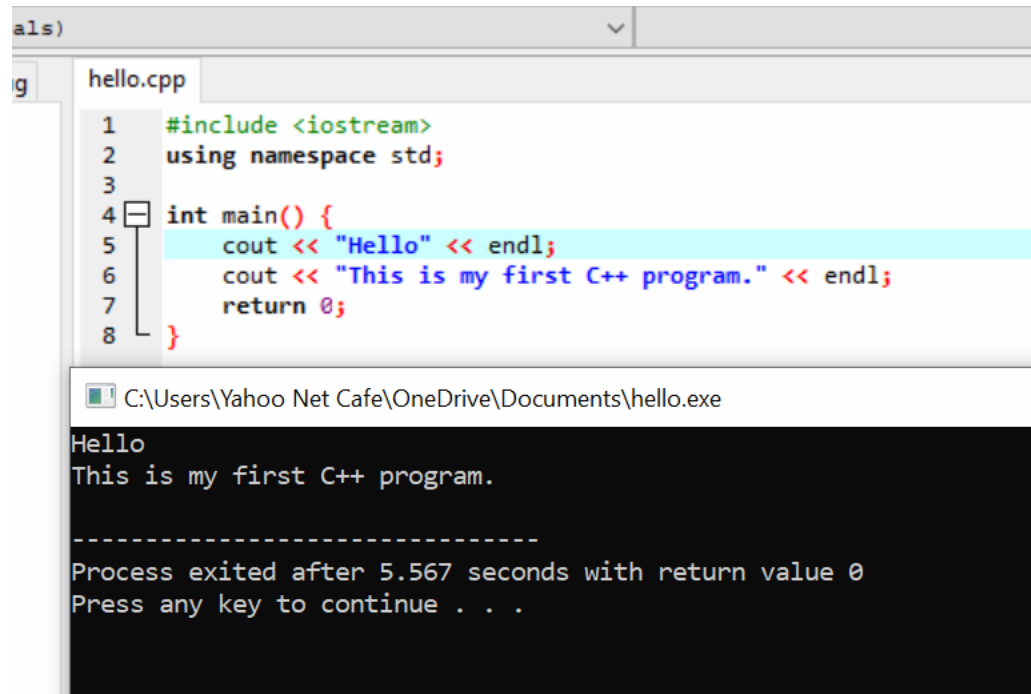
The image shows a code editor window titled 'hello.cpp' containing the following C++ code:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Hello, Compiler!" << endl;
6      cout << "This is my first C++ program." << endl;
7      return 0;
8  }
```

Below the code editor is a terminal window showing the output of the program:

```
C:\Users\Yahoo Net Cafe\OneDrive\Documents\hello.exe
Hello, Compiler!
This is my first C++ program.

-----
Process exited after 6.838 seconds with return value 0
Press any key to continue . . .
```



The image shows a screenshot of a C++ development environment. At the top, a tab labeled 'hello.cpp' is open. The code in the editor is as follows:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Hello" << endl;
6      cout << "This is my first C++ program." << endl;
7      return 0;
8  }
```

Below the code editor, a console window displays the output of the program. The title bar of the console window reads 'C:\Users\Yahoo Net Cafe\OneDrive\Documents\hello.exe'. The output text is:

```
Hello
This is my first C++ program.

-----
Process exited after 5.567 seconds with return value 0
Press any key to continue . . .
```